

M105 — Programmer en JavaScript

Résumé théorique · OFPPT · Filière Développement Digital · 120h

PARTIE 1 — Rôle de JavaScript (12h)

Langage de script vs compilé

Un **compilateur** transforme tout le code en code machine avant exécution (ex: C, C++, C#). Un **interpréteur** traduit ligne par ligne au moment de l'exécution (ex: JS, Python, PHP).

Un **langage de script** est une série de commandes exécutées sans compilation préalable, via un interpréteur.

- Scripts **côté client** : HTML, CSS, **JavaScript** — s'exécutent dans le navigateur.
- Scripts **côté serveur** : PHP, Node.js, Python — s'exécutent sur le serveur.

Architecture Client / Serveur

Le navigateur (client) envoie une requête HTTP à un serveur DNS, qui renvoie l'adresse IP du serveur web. Le serveur répond avec les fichiers HTML/CSS/JS. Le navigateur interprète et affiche la page.

Méthode	Rôle
GET	Récupérer une ressource
POST	Créer / envoyer des données
PUT	Mettre à jour une ressource
DELETE	Supprimer une ressource

Écosystème de développement

IDE recommandé : **Visual Studio Code**. Frameworks front-end : Angular, React, Vue.js, jQuery. Frameworks back-end : Node.js, PHP, Python.

PARTIE 2 — Fondamentaux JavaScript (48h)

Variables & Types de données

Mot-clé	Portée	Réassignable	Usage
var	Function (global)	Oui	Ancien code
let	Block	Oui	Variable standard
const	Block	Non	Constante

Types primitifs : string, number, boolean, undefined, null

Types structurels : Date, Array, Object, Function

```
typeof "Ali"      // "string"
typeof 3.14       // "number"
typeof true       // "boolean"
```

```
typeof undefined // "undefined"
typeof null      // "object"
```

Opérateurs

- Arithmétiques : + - * / % ** ++ --
- Comparaison : == (valeur) === (valeur+type) != !== > < >= <=
- Logiques : && || !
- Type : typeof, instanceof

Affichage & Saisie

- **document.write()** — écrit dans le document HTML
- **window.alert()** — boîte d'alerte
- **console.log()** — console du navigateur (debug)
- **prompt()** — saisie utilisateur

Structures de contrôle

```
// Conditionnelle
if (note >= 10) { ... } else if (note > 8) { ... } else { ... }

// Switch
switch(expr) { case x: ...; break; default: ...; }

// Boucles
for (let i=0; i<5; i++) { ... }
for (let x of tableau) { ... }
for (let key in objet) { ... }
while (condition) { ... }
do { ... } while (condition);
```

Fonctions

```
// Déclaration classique
function maFonction(a, b) { return a + b; }

// Arrow function
const add = (a, b) => a + b;

// Callback
function calcul(n1, n2, callback) {
  callback(n1 + n2);
}
calcul(1, 5, console.log); // affiche 6
```

Promises (Asynchrone)

Une **Promise** a 3 états : **pending** (en attente), **fulfilled** (résolue), **rejected** (rejetée).

```
let p = new Promise((resolve, reject) => {
  if (ok) resolve("Succès");
  else reject("Erreur");
});
```

```
p.then(val => console.log(val))
.catch(err => console.log(err))
.finally(() => console.log("Fin"));
```

Gestion des exceptions

```
try {
  let x = y + 1; // y inexistant
} catch(err) {
  console.log(err.name);    // ReferenceError
  console.log(err.message);
} finally {
  console.log("Toujours exécuté");
}
```

Objets

```
// Syntaxe littérale
const tel = {
  marque: "Samsung",
  prix: 400,
  verifierStock: function() { return this.stock > 0; }
};

// Constructeur
function Telephone(marque, prix) {
  this.marque = marque;
  this.prix = prix;
}
let t1 = new Telephone("Samsung", 400);
```

Tableaux (Array)

```
let tab = ['A', 'B', 'C'];
tab.push('D');           // ajoute fin
tab.unshift('Z');        // ajoute debut
tab.pop();               // supprime fin
tab.shift();             // supprime debut
tab.splice(1, 1);        // supprime index 1
tab.sort();              // tri alphabetique
tab.indexOf('B');        // retourne 1
```

JSON

Format d'échange de données léger, compatible avec tous les langages modernes.

```
let json = '{"nom":"Ali","age":25}';
let obj  = JSON.parse(json);    // JSON -> Objet JS
let str  = JSON.stringify(obj); // Objet JS -> JSON
```

Expressions régulières (Regex)

```
let regex = /[A-Z]/;
let texte = "cours JavaScript";
```

```
texte.search(regex);          // 6 (index de J)
texte.replace(/js/i, "JS"); // insensible casse
texte.match(/[a-z]+/g);       // tableau des mots
```

PARTIE 3 — Manipulation du DOM (30h)

Le **DOM** (Document Object Model) représente la page HTML sous forme d'arbre d'objets. JavaScript peut modifier dynamiquement tout contenu, style et structure.

Sélection d'éléments

```
document.getElementById("id1")          // par ID
document.getElementsByClassName("classe") // par classe
document.getElementsByTagName("p")       // par balise
document.querySelector("#id1")          // 1er correspondant
document.querySelectorAll(".classe")    // tous correspondants
```

Navigation dans l'arbre

- **parentNode / parentElement** — accéder au parent
- **childNodes / children** — liste des enfants
- **firstChild / firstElementChild** — premier enfant
- **lastChild / lastElementChild** — dernier enfant
- **nextElementSibling / previousElementSibling** — frères

Manipulation des éléments

```
// Créer et ajouter
let p = document.createElement("p");
p.innerHTML = "Nouveau paragraphe";
document.getElementById("parent").append(p);

// Modifier
p.textContent = "Texte modifié";
p.style.color = "red";
p.setAttribute("class", "highlight");

// Supprimer
parent.removeChild(p);
p.removeAttribute("class");
```

PARTIE 4 — Événements Utilisateur (12h)

addEventListener

```
let btn = document.getElementById("btn");
btn.addEventListener("click", function() {
  alert("Cliqué !");
});

// Supprimer
```

```
btn.removeEventListener("click", maFonction);
```

Événements souris

- **click** — clic simple
- **dblclick** — double clic
- **mouseover / mouseout** — entrée/sortie de l'élément
- **mouseenter / mouseleave** — similaire, sans propagation
- **mousemove** — déplacement du curseur
- Propriétés : **event.clientX/Y** (position dans la fenêtre), **event.button** (bouton pressé)

Événements clavier

- **keydown** — touche enfoncée
- **keypress** — caractère pressé
- **keyup** — touche relâchée

```
input.addEventListener("keydown", (e) => {  
  console.log(e.key); // "a", "Enter"...  
  console.log(e.code); // "KeyA", "Enter"...  
});
```

Formulaires

```
// Soumettre  
document.getElementById("form1").submit();  
  
// Empêcher soumission  
form.addEventListener("submit", (e) => {  
  e.preventDefault();  
});  
  
// Validation JS  
if (x == "") { alert("Champ requis"); return false; }
```

Validation HTML5 : attributs **required**, **pattern**, **min**, **max**, **type**.

PARTIE 5 — jQuery & AJAX (18h)

jQuery — Introduction

Bibliothèque JS open-source (John Resig, 2006). Simplifie la manipulation du DOM, les événements, les animations et AJAX.

```
// Intégration CDN  
  
// Syntaxe de base  
$(selecteur).action()  
  
// Attendre le chargement du DOM  
$(document).ready(function() {  
  $("p").hide();  
  $("#btn").click(() => $("p").toggle());  
});
```

```
});
```

Sélecteurs & Actions jQuery

- `$(this).hide()` — masquer l'élément courant
- `$("#p").css("color","red")` — changer le style
- `$(".classe").html("texte")` — modifier le contenu
- `$("#id").attr("href", "url")` — modifier un attribut
- Chaînage : `$("#p1").css("color","red").slideUp(2000).slideDown(2000)`

AJAX — Introduction

AJAX (Asynchronous JavaScript And XML) permet d'envoyer/recevoir des données serveur **sans recharger la page**.

Basé sur l'objet **XMLHttpRequest** (XHR) avec les états `readyState` 0→4 (0=non init, 4=réponse prête).

```
// XMLHttpRequest natif
let xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
        document.getElementById("demo").innerHTML = this.responseText;
    }
};
xhttp.open("GET", "data.txt", true);
xhttp.send();
```

AJAX via jQuery

Méthode	Description
<code>\$.ajax(url, options)</code>	Requête complète avec toutes les options
<code>\$.get(url, callback)</code>	Requête GET asynchrone
<code>\$.post(url, data, callback)</code>	Requête POST asynchrone
<code>\$.getJSON(url, callback)</code>	GET avec réponse JSON
<code>\$('#div').load(url)</code>	Charger du HTML depuis le serveur

```
// Exemple $.ajax()
$.ajax("/api/data", {
    type: "GET",
    dataType: "json",
    success: function(data) { console.log(data); },
    error: function(err) { console.log("Erreur:", err); }
});

// Exemple $.post()
$.post("/api/submit",
    { nom: "Ali", age: 25 },
    function(response) { console.log(response); }
);
```